An algebraic model for the representation of accounting systems

Robert A. Nehmer
Campbell School of Business, Berry College, Mount Berry, GA 30149–5624, USA

Derek Robinson
Department of Mathematics, University of Illinois-Urbana, Urbana, IL 61801, USA

The paper describes an algebraic structure which embodies the essential features of the double-entry accounting system. The structure has the benefits of providing reliable means to record the balances of the accounts of the system and to apply transactions to the accounts. It will detect transactions which are of an undesirable type or which lead to inadmissible balances, thus preserving the integrity of the system. The stucture is also able to generate reports and includes procedures to verify whether an existing balance has been obtained by legitimate transactions. Finally, it provides methods for comparing accounting systems with one another and over time.

1. Introduction

The object of the present work is to describe an algebraic structure which embodies the essential features of the double-entry accounting system. Such an algebraic structure has the benefits of providing reliable means to record the debit or credit balances of the various accounts of the system and apply transactions to the system. It will detect transactions which are of an undesirable type or which lead to inadmissable balances, thus preserving the integrity of the system. The structure is also able to generate reports (formal or ad hoc) on the status of various sets of accounts including, but not limited to, balance sheets, the income statement and the statement of cash flows, that is, it must provide useful information to management decision makers. The structure includes procedures to verify whether an existing balance has been obtained by legitimate transactions and is thereby auditable. Finally, it provides the ability to compare different accounting systems and to classify changes that ordinarily occur in the evolution of an accounting system.

The benefits from the successful construction of such a structure include a close examination and better understanding of the mathematical nature of accounting

© J.C. Baltzer AG, Science Publishers

systems. Adopting the precise methods and language of abstract algebra will be a powerful aid in the analysis of accounting systems. This benefit can be seen directly in three areas. First, the algebraic structure provides a precise mathematical description of the double-entry bookkeeping system which underlies all business practice. Secondly, since essential elements of all businesses can be represented in their corresponding algebraic structures, the method provides a way to compare different accounting systems based on differences in their underlying algebraic structures which provides the mathematical grounding for analyses based on computational complexity, a subject left to future research. Thirdly, the method allows the introduction of control features, again representable in terms of abstract algebra, which model some of the control features of a typical business enterprise and can be used in evaluating the strengths and weaknesses of those features vis-a-vis the firm which the structure represents. The control features provided by the structure are not all encompassing, but they would disallow a number of improper transactions from entering the system. The system presented below is a representation of the conceptual core of an accounting systems. Any realistic implementation of it would of course require administrative and behavioral control systems as well.

In the point of view adopted here, the state of an accounting system at any instant is described by a column vector whose entries are integers: positive, negative or zero. The ith entry of this balance vector is the balance of the ith account. Here it is convenient to represent a debit balance by a positive integer and a credit balance by a negative integer since this forces most typical asset balances to be positive and most liability and equity accounts to be negative. It is also convenient to make all balances integers, so amounts are considered as being either in cents or in whole dollars. Since the system is double-entry bookkeeping, the sum of the entries in the balance vector will always be zero, reflecting the fact that the system of accounts must always be in balance.

A basic concept used throughout this analysis is a transaction. This is a function that operates on the existing balance vector to produce a new balance vector, by adding a positive or negative number to each account involved in the transaction. It is natural to represent a transaction as a vector whose entries sum to zero and which is of the same dimension as the balance vector. A transaction is then thought of as the vector addition of the transaction vector to the balance vector. In many cases, a transaction will only involve an exchange of funds between two accounts, in which event, the corresponding transaction vector will have just two non-zero entries, one being the negative of the other. These will be called simple transactions. More complex transactions, however, will occur as, for example, in the sale of an asset where there will be more than one positive and/or negative entry.

In a real-life system, there would be certain transactions which would be regarded as improper: for example, credit pension fund and debit equity account. There should therefore be a list of allowable transactions, those that can actually be

applied to the system. However, even if a transaction is allowable, it might still be rejected if it caused an unacceptable balance in one of the accounts. For example, customer accounts are likely to have credit limits. Thus, the algebraic structure recognizes the existence of both allowable transactions and allowable balances. It models for these types of integrity issues.

A system of the type described can be conveniently visualized as an automaton which checks each incoming transaction to see if it is allowable; if so, it computes the new balance vector and checks to see if this too is allowable. If this is the case, it records the new balance. The states of the automaton are thus the allowable balance vectors of the system. The input alphabet consists of all transaction vectors (allowable or not) and the transition function is a transition from one allowable balance vector to another allowable balance function. Note that the transition functions are not quite the same as the transaction functions since a transaction may map an allowable balance vector to one which is not allowed, in which event the corresponding transaction function would cause no change in the balance vector. The specific implementation of balances and transactions is provided in section 2.

There is a standard procedure which associates with an automaton an algebraic structure called a monoid. A monoid is a set M with a distinguished element e, the identity, and a rule for combining elements of M. This rule should satisfy the associative law and combining an element with e through this rule should leave the element unchanged. For this reason, the structure of an accounting system described here gives rise to a monoid. It is not always the case that the inverse of an allowable transaction is allowable. In accounting systems where this is the case, however, the corresponding monoid will have the additional property that each element x in M has an inverse. A monoid with this property is called a group. These issues are addressed at length in section 3.

Section 4 begins the discussion of how to produce useful information from the system. In order to produce a report on some aspect of an accounting system, one can partition the accounts into equivalence classes. Accounts belonging to the same subset are regarded as equivalent and are combined to form a single account. The balance of the new account is the sum of the balances of the equivalent accounts. The same procedure can be applied to the transaction vectors. The effect of such a procedure is to produce a new accounting system known as a quotient of the original system. The balance vectors of the original system would then feature in a report on the status of the various sets of accounts that have been combined. The process of forming a quotient structure from a given structure and an equivalence relation is a familiar phenomenon in abstract algebra.

The simplest instance of a quotient system is when the set of accounts is partitioned into asset accounts, liability accounts, and equity accounts. The resulting quotient system has three accounts: total assets, total liabilities, and total equity. There is a natural function from the accounting system to any quotient of the system. This function is an important example of a homomorphism between accounting systems.

In sections 5 and 6, the general notion of a homomorphism will be shown below to give an efficient way of classifying changes in accounting systems.

In section 7, the problem of balance verification is addressed. What is required here is an effective procedure to test if a purported final balance vector could actually have arisen in an accounting system by legitimate operations. Accordingly, one must decide if there is a finite sequence of allowable transactions which, when applied to the original balance vector, will lead to the final balance vector. The sequence should be such that all of the intermediate balance vectors are allowable. The existence of such a verification procedure clearly enhances the auditibility of the system.

Section 8 shows that for an important class of accounting systems, including most that arise in practice, such an effective procedure does exist. For a given final balance vector, the mathematical problem is to find out if it could be the solution to a system of linear equations with integral coefficients. This is a problem of integer programming and there are well-established algorithms to solve it. It will be shown that these finitely specified accounting systems, which are the closest to real-world accounting systems, are so formulated as to permit computer implementation in an appropriate language.

2. Balances and transactions

In this section, the fundamentals of the algebraic representation of accounting systems are presented. Let n be a positive integer which corresponds to the number of accounts in such a system. Let $\mathbb{Z}^n$ denote the set of all $n \times 1$ column vectors, $\boldsymbol{v}$, with integral entries v_i. Thus,

$$\boldsymbol{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}, \quad v_i \in \mathbb{Z}.$$

Elements of $\mathbb{Z}^n$ can be added by matrix addition, i.e., by adding corresponding entries. Thus, if $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{Z}^n$, then $\boldsymbol{u} + \boldsymbol{v}$ has ith entry $u_i + v_i$. Both the commutative and the associative laws hold for elements of $\mathbb{Z}^n$ under addition. Furthermore, the zero vector, $\mathbf{0}$, all of whose entries equal zero, provides an identity element to the structure. Finally, $\boldsymbol{u} + (-\boldsymbol{u}) = \mathbf{0}$, where $-\boldsymbol{u}$ is the vector whose ith entry is $-u_i$. The validity of these laws means that the pair of objects $(\mathbb{Z}^n, +)$ form a commutative (or abelian) group. In fact, this is isomorphic to the additive group of all the integers or a free abelian group of rank n.

A vector $\boldsymbol{v}$ in $\mathbb{Z}^n$ such that $\sum_{i=1}^{n} v_i = 0$ is called a balance vector. One observes easily that the subset of all balance vectors in $\mathbb{Z}^n$ is closed under addition, the formation of negatives, and contains the zero vector. Thus, the set of all balance vectors, which will be written Bal(n), is a subgroup of $\mathbb{Z}^n$ which has rank $n - 1$; as the entries of a balance vector are subject to a single linear condition, there are $n - 1$ degrees of freedom. Sometimes, Bal(n) will be referred to as the balance space.

2.1. Transactions

Let $\boldsymbol{v}$ be a given balance vector in $\mathbb{Z}^n$. If $\boldsymbol{x} \in \text{Bal}(n)$, then $\boldsymbol{x} + \boldsymbol{v} \in \text{Bal}(n)$, so there is a function

$$t_{\boldsymbol{v}} : \text{Bal}(n) \to \text{Bal}(n)$$

given by $t_{\boldsymbol{v}}(\boldsymbol{x}) = \boldsymbol{x} + \boldsymbol{v}$. Call $t_{\boldsymbol{v}}$ the transaction determined by the balance vector $\boldsymbol{v}$. Denote the set of all transaction vectors by Trans(n); thus

$$\text{Trans}(n) = \{t_{\boldsymbol{v}} | \ \boldsymbol{v} \in \text{Bal}(n)\}.$$

Now transactions, being functions on the balance space, can be combined by functional composition. Then Trans(n) is an abelian group with respect to the binary operation of composition since

$$t_{\boldsymbol{u}} \circ t_{\boldsymbol{v}} = t_{\boldsymbol{u}+\boldsymbol{v}} = t_{\boldsymbol{v}} \circ t_{\boldsymbol{u}},$$

a law which also shows that the function $\boldsymbol{v} \to t_{\boldsymbol{v}}$ is an isomorphism from the group Bal(n) to the group Trans(n). An isomorphism is a function which both maps members of one group to members in another in a one-to-one correspondence and also maps all elements from each group to one another. Thus, these groups are isomorphic, Bal(n) $\simeq$ Trans(n), and the isomorphism is the regular representation of Bal(n) in the sense that it is the smallest representative set.

Isomorphic groups have the same properties, from an algebraist's point of view. Thus, one can simplify the notation by identifying $\boldsymbol{v}$ with $t_{\boldsymbol{v}}$ and Bal(n) with Trans(n). Of course, the price to pay for this notational convenience is a degree of ambiguity. But in practice there is little danger of confusion, especially if one speaks of "the balance vector $\boldsymbol{v}$" or "the transaction vector $\boldsymbol{v}$". This point of view will be adopted henceforth.

2.2. Simple transactions

The most primitive transaction, which does not affect the balance of any account, is the trivial transaction which corresponds to the zero vector $\boldsymbol{0}$. Since a transaction cannot have just one non-zero entry, the next type to consider is where there are exactly two non-zero entries, one being the negative of the other. These are called simple transactions. Define $\boldsymbol{e}(i, j)$ to be the $n \times 1$ column vector whose ith entry is 1, jth entry –1 and all other entries zero. Thus, a simple transaction has the form $a\boldsymbol{e}(i, j)$ for a positive integer a and some i and j. The importance of this can be seen from the following proposition.

Proposition. Every transaction vector $\boldsymbol{v}$ on n accounts is uniquely expressible in the form $\boldsymbol{v} = a_1\boldsymbol{e}(1, 2) + a_2\boldsymbol{e}(2, 3) + \cdots + a_{n-1}\boldsymbol{e}(n-1, n)$. Here, the coefficients are given by the formula $a_i = \sum_{j=1}^{i} v_j$.

Proof. One verifies by vector addition that the sum of the vectors $a_1\boldsymbol{e}(1, 2),\ldots,$ $a_{n-1}\boldsymbol{e}(n-1, n)$ with a_i as above is in fact $\boldsymbol{v}$. One has to keep in mind that v_n equals $(-v_1) + (-v_2) + \cdots + (-v_{n-1})$ since $\boldsymbol{v}$ is a balance vector. For example,

$$\begin{bmatrix} a_1 \\ a_2 \\ -a_1 - a_2 \end{bmatrix} = a_1\begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} + (a_1 + a_2)\begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix}.$$

The uniqueness for the expression of $\boldsymbol{v}$ follows from the linear independence of the vectors $\boldsymbol{e}(i, i+1)$. □

The proposition asserts that Trans(n) is a free abelian group with basis $\boldsymbol{e}(1, 2), \boldsymbol{e}(2, 3),\ldots, \boldsymbol{e}(n-1, n)$.

2.3. Transaction types

General transactions can be classified according to the distribution of positive, negative or zero entries. Let $\boldsymbol{v} \in \mathrm{Bal}(n)$; then the type, $\boldsymbol{t}(\boldsymbol{v})$, of $\boldsymbol{v}$ is defined to be the $n \times 1$ column vector whose ith entry is +, – or 0 according to whether $v_i > 0$, $v_i < 0$, or $v_i = 0$. If $n = 1$, there is just one possible type, [0]. For $n = 2$, there are three types:

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} + \\ - \end{bmatrix}, \quad \begin{bmatrix} - \\ + \end{bmatrix}.$$

and there are in fact 13 types when $n = 3$. Generally, there will be $3^n - 2^{n+1} + 2$ types for a system with n accounts since 3^n equals all combinations of $\{0, +, -\}$, $2^n - 1$ equals all non-allowable combinations $\{0, +\}/-$ or $\{0, -\}/+$ and $3^n + 2(2^n - 1) = 3^n - 2^{n+1} + 2$.

There is a natural ordering of types. If $\boldsymbol{u}, \boldsymbol{v} \in \mathrm{Bal}(n)$, let

$$\boldsymbol{t}(\boldsymbol{u}) \leq \boldsymbol{t}(\boldsymbol{v})$$

mean that $|u_i| \leq |v_i|$ for $i = 1, 2,\ldots, n$. Thus, the zero type appears at the lowest level in the ordering, while simple transaction types appear at the next lowest level. The relation ≤ is a partial order on the set of all types, Type(n). The pair (Type(n), ≤) is a partially ordered set (or poset). Thus, there is a hierarchy of transaction types in which transactions at higher levels of the poset are more complicated.

In practice one would expect most transactions to occur at a low level. Nevertheless, certain transaction types, such as the closing process, especially if it is considered as one transaction, are quite high in the poset ordering. One can also conceive of transactions which would affect every account, perhaps during a reorganization. Thus, there could be transactions occurring at arbitrarily high levels of the poset of types.

3. The definition of an accounting system

After the discussion of the preliminary notions of balance and transaction, it is now time to give a precise definition of an abstract accounting system. Let n be a positive integer and let $A = \{a_1, a_2, \ldots, a_n\}$ be an ordered set of n distinct elements, called the accounts. An accounting system on the set A is a triple $\mathcal{A} = (A|T|B)$, where T and B are subsets of Bal(n) called, respectively, the sets of allowable transactions and allowable balances. (Recall Bal(n) and Trans(n) are isomorphic.)

The mode of operation of the system $\mathcal{A}$ can be conveniently visualized by means of an automaton, $\hat{\mathcal{A}}$. The input set, state set, and output set of this automaton are Bal(n), B and $B \cup \{*\}$, repectively, where the symbol $*$ stands for ERROR. The transition function $\delta : B \times \text{Bal}(n) \to B$ and the next output function $\lambda : B \times \text{Bal}(n) \to B \cup \{*\}$ are given by

$$\delta(\boldsymbol{b}, \boldsymbol{v}) = \begin{cases} \boldsymbol{b} + \boldsymbol{v} & \text{if } \boldsymbol{v} \in T \text{ and } \boldsymbol{b} + \boldsymbol{v} \in B, \\ \boldsymbol{b} & \text{otherwise;} \end{cases}$$
$$\lambda(\boldsymbol{b}, \boldsymbol{v}) = \begin{cases} \boldsymbol{b} + \boldsymbol{v} & \text{if } \boldsymbol{v} \in T \text{ and } \boldsymbol{b} + \boldsymbol{v} \in B, \\ * & \text{otherwise.} \end{cases} \tag{1}$$

To see how the automaton operates, assume that it is in state $\boldsymbol{b}$, so $\boldsymbol{b}$ is the present balance vector. A transaction $\boldsymbol{v}$ is fed into the system and read by the automaton. If $\boldsymbol{v} \in T$, i.e., $\boldsymbol{v}$ is an allowable transaction, then the vector $\boldsymbol{b} + \boldsymbol{v}$ is computed. If the result is an element of B, i.e, the balance vector $\boldsymbol{b} + \boldsymbol{v}$ is allowable, then $\boldsymbol{b} + \boldsymbol{v}$ becomes the next state, the new balance vector. The new balance is then printed on the output tape. If either condition is not met, the automaton remains in state $\boldsymbol{b}$ and prints ERROR on the output tape. Notice that the input set of an automaton is usually taken to be a finite set. However, this is not a problem in the present context since the vectors of integers can be represented by a finite string of 0's and 1's with separation symbols.

3.1. The monoid of an accounting system

A standard example of a monoid is the set of all functions $\alpha : S \to S$, where S is a fixed non-empty set and the operation is the composition of functions; this monoid is written Fun(S). There is a well-known procedure for associating a monoid with an automaton which may be applied to an accounting system $\mathcal{A} = (A|T|B)$. The automaton $\hat{\mathcal{A}}$ has the transition function δ given by (1) above. Let $\boldsymbol{v} \in \text{Bal}(n)$. Define a function $\hat{\boldsymbol{v}} : B \to B$ by the rule

$$\hat{\boldsymbol{v}}(\boldsymbol{b}) = \delta(\boldsymbol{b}, \boldsymbol{v}),$$

where $\boldsymbol{b} \in B$. Then $\hat{\boldsymbol{v}}$ is an element of the monoid Fun(B). The monoid of the system $\mathcal{A}$, Mon($\mathcal{A}$), is defined to be the submonoid of Fun(B) generated by all of the $\hat{\boldsymbol{v}}$, $\boldsymbol{v} \in \text{Bal}(n)$. This means that Mon($\mathcal{A}$) consists of all of the composites of finite strings

of $\hat{\boldsymbol{v}}$'s, where $\boldsymbol{v} \in \text{Bal}(n)$. Notice that $\hat{\boldsymbol{v}}$ will be the identity function unless $\boldsymbol{v} \in T \cap \{B_i - B_j\}$, where $B = \{\boldsymbol{x} - \boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{y} \in B\}$ since a valid transaction is representable as the ending balance minus the beginning balance. Thus, Mon($\mathcal{A}$) can equally well be generated by all $\hat{\boldsymbol{v}}$ with $\boldsymbol{v} \in T \cap \{B - B\}$.

The significance of Mon($\mathcal{A}$) is that its elements record transactions that could be obtained as composite strings of allowable transactions. It is evident that the monoid of $\mathcal{A}$ will be non-commutative in general because the order in which a sequence of allowable transactions is applied affects the intermediate balance vectors that appear, some of which may not be allowable.

Examples

(i) $\mathcal{A} = (A|T|\text{Bal}(n))$.

Here there are no restrictions on balances. Any transaction in T will be accepted by the system and the new balance computed by adding the transaction vector to the original balance vector. In this case, $\hat{\boldsymbol{u}} \circ \hat{\boldsymbol{v}} = (\boldsymbol{u} \hat{+} \boldsymbol{v}) = (\boldsymbol{v} \hat{+} \boldsymbol{u}) = \hat{\boldsymbol{v}} \circ \hat{\boldsymbol{u}}$. Hence, the monoid is commutative.

(ii) $\mathcal{A} = (A|\text{Bal}(n)|\text{Bal}(n))$.

Every balance vector and transaction vector is allowable here, and thus this situation will be called the Universal Accounting System on A. Note that its monoid is exactly Trans(n), the abelian group of all transaction vectors regarded as functions on Bal(n). One would not consider this accounting system as a realistic one.

(iii) Let m_i, M_i denote integers or $\pm\infty$ such that $m_i \leq M_i$ for $i = 1, 2, \ldots, n$. Define B to be the set of all n-dimensional balance vectors $\boldsymbol{v}$ such that $m_i \leq v_i \leq M_i$ for $i = 1, 2, \ldots, n$. One can then consider an accounting system $\mathcal{A} = (A|T|B)$ where T is arbitrary. Of course if $m_i = -\infty$ and $M_i = +\infty$, there is no restriction on the balance of the ith account. Otherwise, there will be a limit on the debit or credit balance. In practice, one would expect that m_i and M_i would both be finite for almost every accounting system. If m_i and M_i are finite for all i, then $\mathcal{A}$ will be called bounded. Note that B will also be finite in this event.

(iv) $\mathcal{A} = (A|T|\text{Bal}(n))$, where T is closed under the formation of inverses. This means that if $\boldsymbol{v}$ is allowable, then so is its inverse $-\boldsymbol{v}$. For this system, the associated monoid contains an inverse for each of its elements and is therefore a group. It is in fact an abelian group since there are no restrictions on balances here. One would not expect systems of this type to occur frequently in practice. It is easy to think of transactions that would be acceptable but whose inverses would not, for example, a transfer of funds from cash to the employees pension fund.

(v) For any accounting system which could be conceivably machine operated, there must exist effective procedures to decide if a given transaction or balance vector is allowable. This requires that the system $\mathcal{A} = (A|T|B)$ has to be such that the subsets T and B are recursive. For a set to be recursive, its characteristic function must be Turing machine computable, i.e., there must exist a Turing machine which can compute its characteristic function. Given these facts, it is reasonable to call the system $\mathcal{A}$ recursive if T and B are recursive sets. Thus, the recursive systems are those that could, at least in principle, be machine operated. That there are non-recursive accounting systems is shown by the existence of non-recursive subsets of $\mathbb{Z}$, and hence of Bal(n). Such systems are, of course, of only theoretical interest.

4. Sub-accounting and quotient-accounting systems

In the study of algebraic structures there are certain concepts which occur at an early stage. One of these is the notion of a sub-structure, which is, roughly speaking, a structure contained as a subset in a larger structure of the same type. Subgroups and submonoids are examples of substructures.

Less obvious is the notion of a quotient structure. The idea of a quotient structure is to combine or identify certain elements of a structure in such a way as to produce a new structure. This arises in accounting systems when the balances of certain accounts are combined as, for example, in the process of defining a report.

4.1. Sub-accounting systems

Consider two accounting systems $\mathcal{A}_1 = (A_1|T_1|B_1)$ and $\mathcal{A}_2 = (A_2|T_2|B_2)$, where $A_1 \subseteq A_2$ and the order of accounts in A_1 must be consistent with the order in A_2 in that the accounts in A_2 which also appear in A_1 must occur in the same relative order as in A_1. Suppose that if zero entries are inserted in vectors of T_1 and B_1 for accounts in A_2 but not in A_1, the resulting vectors are in T_2 and B_2, respectively. Then $\mathcal{A}_1$ is called a sub-accounting system of $\mathcal{A}_2$. Thus, any transaction or balance vector allowed in $\mathcal{A}_1$ would correspond to an allowable vector in $\mathcal{A}_2$. For example, any accounting system on a set of accounts A is a sub-accounting system of the universal accounting system on A.

4.2. Quotient accounting systems

Let $\mathcal{A} = (A|T|B)$ be an accounting system with account set $A = \{a_1, a_2, \ldots, a_n\}$. The intent of the ensuing construction is to permit certain accounts to be amalgamated by combining their balance and transaction entries. The starting point for the quotient construction is an equivalence relation, E, on the set A, i.e., a binary relation on A which is reflexive, symmetric and transitive. The relation E can always be thought of

as a partition of A, that is, an expression for A as the union of disjoint non-empty subsets where the subsets are the equivalence classes. Thus, E splits the set A into, say, n' equivalence classes, where $n' \leq n$. There is an induced ordering of the equivalence classes: in each equivalence class, select the account with the smallest subscript and order the equivalence classes by these chosen representatives. Let A' be the set of equivalence classes so ordered.

The set A' is to be the account set of the quotient system. To complete the definition, the sets of allowable transactions and balances, T' and B', must be specified. If $\boldsymbol{v} \in \mathrm{Bal}(n)$, define a balance vector $\boldsymbol{v}'$ in $\mathrm{Bal}(n')$ by making the ith entry of $\boldsymbol{v}'$ equal to the sum of all the entries v_j, where j is such that $a_j E a_i$. Therefore,

$$v_i' = \sum_{a_j E a_i} v_j .$$

Thus, the balances of accounts equivalent under E are to be combined. Notice that

$$\sum_{i=1}^{n'} v_i' = \sum_{i=1}^{n} v_i = 0,$$

so that $\boldsymbol{v}'$ really belongs to $\mathrm{Bal}(n')$. It is also easy to show that $(\boldsymbol{u} + \boldsymbol{v})' = \boldsymbol{u}' + \boldsymbol{v}'$. This expresses the fact that the surjective function $\boldsymbol{v} \to \boldsymbol{v}'$ is a group homomorphism from $\mathrm{Bal}(n)$ to $\mathrm{Bal}(n')$.

The allowable sets T' and B' can now be defined as

$$T' = \{\boldsymbol{v}' \mid \boldsymbol{v} \in T\} \quad \text{and} \quad B' = \{\boldsymbol{v}' \mid \boldsymbol{v} \in B\}.$$

The quotient accounting system of $\mathcal{A}$ with respect to the equivalence relation E is defined to be $\mathcal{A}/E = (A'|T'|B')$. Thus, the allowable vectors in the quotient system are just the vectors which arise from the allowable vectors in $\mathcal{A}$ when vector entries are summed for all accounts that are equivalent under E.

A very simple example of a quotient system arises when the accounts of a system $\mathcal{A} = (A|T|B)$ are classified as asset accounts, liability accounts, and equity accounts after the close of a period. The resulting quotient system will then have three accounts corresponding to total assets, total liabilities and total equity.

The quotient systems of an accounting system $\mathcal{A}$ correspond to equivalence relations or partitions of the set $\{a_1, a_2, \ldots, a_n\}$ of all accounts. There is a well-known lattice structure on the set of all equivalence relations on a set. Here, a lattice is a poset with least upper bounds and greatest lower bounds. The partial order on equivalence relations is just set inclusion, while the least upper bound of the equivalence relations E_1 and E_2 is the equivalence closure of their union and the greatest lower bound is their intersection.

Thus, there is a hierarchy of quotient systems of $\mathcal{A}$ corresponding to the lattice of equivalence relations or partitions. At the lowest level is the identity relation wherein an account is equivalent only to itself. At the highest level is the universal

relation in which all accounts are equivalent. The corresponding quotient systems of $\mathcal{A}$ are $\mathcal{A}$ itself and the trivial accounting system with one account and a perpetually zero balance. The total number of quotients of an n-account system $\mathcal{A}$ equals the number of partitions of the set $\{a_1, a_2,\ldots, a_n\}$. The latter is a complicated function of n, depending on the number of partitions of the integer n. Of course, relatively few of the corresponding quotients are of interest from the accounting point of view.

5. Homomorphisms of accounting systems

In algebra, a basic research method is to study homomorphisms between structures of the same type. These are functions which relate the algebraic operations of the structures. Following this well-established procedure, homomorphisms of accounting systems are now introduced.

Let $\mathcal{A} = (A|T|B)$ and $\mathcal{A}^* = (A^*|T^*|B^*)$ be two accounting systems with account sets $A = \{a_1, a_2,\ldots, a_n\}$ and $A^* = \{a_1^*, a_2^*,\ldots, a_{n'}^*\}$. To define a homomorphism from $\mathcal{A}$ to $\mathcal{A}^*$, one starts with a function

$$\Theta : A \to A^*.$$

Θ must send each account of $\mathcal{A}$ to an account in $\mathcal{A}^*$. The function Θ is now used to produce a new function,

$$\Theta : \mathrm{Bal}(n) \to \mathrm{Bal}(n^*),$$

given by the rule

$$(\Theta(\boldsymbol{u}))_i = \sum_{\Theta(a_j) = a_i^*} u_j^*.$$

Here, the convention is that $(\Theta(\boldsymbol{u}))_i$ is zero if Θ does not map any a_j to a_i^*, i.e., a_i^* does not belong to the image of Θ. Thus, account a_i^* receives as its balance the sum of the balances of all accounts a_j which Θ maps to a_i^*; if there is no such account, then a_i^* receives a zero entry. It is easy to verify that $\Theta(\boldsymbol{u} + \boldsymbol{v}) = \Theta(\boldsymbol{u}) + \Theta(\boldsymbol{v})$, which simply says that Θ is a group homomorphism from $\mathrm{Bal}(n)$ to $\mathrm{Bal}(n^*)$.

One can now form the images of T and B under Θ as

$$\Theta(T) = \{\Theta(\boldsymbol{v}) | \boldsymbol{v} \in T\} \quad \text{and} \quad \Theta(B) = \{\Theta(\boldsymbol{v}) | \boldsymbol{v} \in B\}.$$

These, of course, are subsets of $\mathrm{Bal}(n^*)$. If it happens that $\Theta(T) \subseteq T^*$ and $\Theta(B) \subseteq B^*$, then Θ is called a homomorphism from $\mathcal{A}$ to $\mathcal{A}^*$. Certain types of homomorphisms are of special interest. Θ is called a monomorphism if the original function Θ is injective in that distinct accounts in A have distinct images in A^* under Θ. Next, Θ is said to be an epimorphism if Θ is surjective and $\Theta(T) = T^*$, $\Theta(B) = B^*$. This means that each account and each allowable transaction and balance vector of $\mathcal{A}^*$ must be the image under Θ or Θ of a corresponding account or vector of $\mathcal{A}$.

Finally, Θ is said to be an isomorphism if it is both a monomorphism and an epimorphism. Thus, an isomorphism establishes a one-to-one correspondence between

the accounts and allowable transactions and balances of $\mathcal{A}$ and the same entities of $\mathcal{A}^*$. Two accounting systems $\mathcal{A}$ and $\mathcal{A}^*$ are isomorphic if there exists an isomorphism from $\mathcal{A}$ to $\mathcal{A}^*$. The idea here is that isomorphic accounting systems operate in exactly the same manner, even though they may have a quite different set of accounts.

5.1. Examples of homomorphisms

Next, a number of examples will be furnished which are designed to show how the abstract notion of a homomorphism underlies several natural operations applicable to accounting systems.

(i) Changing the order of the accounts.

The accounts in an accounting system $\mathcal{A} = (A|T|B)$ are written in some definite order $a_1, a_2, \ldots, a_n$. Consider a change in the ordering; this amounts to forming a permutation Π of the set $\{1, 2, \ldots, n\}$ and taking the new order to be $a_{\Pi(1)}, a_{\Pi(2)}, \ldots, a_{\Pi(n)}$. One can also regard Π as a function from A to A in which $a_i \to a_{\Pi(i)}$. Let T^* and B^* denote the arising sets of vectors when the vectors of T and B have their entries permuted according to Π. For example, a typical vector in T^* would be of the form $\boldsymbol{v}^*$, where $\boldsymbol{v} \in T$ and the ith entry of $\boldsymbol{v}^*$ is $v_{\Pi^{-1}(i)}$. Then $\mathcal{A}^* = (A|T^*|B^*)$ is a new accounting system and $\Pi : A \to A^*$ induces an isomorphism $\Pi : A \to A^*$. There is nothing mysterious occurring here, merely a change in the order of accounts, balance entries and transaction entries.

(ii) Addition of accounts, transactions and balances.

Another natural operation that may be applied to an accounting system is the addition of further accounts, allowable transactions and allowable balances. Suppose that $\mathcal{A}$ is a sub-accounting system of a system $\mathcal{A}^*$. Thus, $\mathcal{A} = (A|T|B)$ and $\mathcal{A}^* = (A^*|T^*|B^*)$, where $A \subseteq A^*$. Now the set inclusion function $i : A \to A^*$ induces an injective function $i : \mathrm{Bal}(n) \to \mathrm{Bal}(n^*)$. Moreover, it follows from the definition of a sub-accounting system that $i(T) \subseteq T^*$ and $i(B) \subseteq B^*$. Thus, $i : \mathcal{A} \to \mathcal{A}^*$ is a monomorphism which we will call the inclusion monomorphism from $\mathcal{A}$ to $\mathcal{A}^*$. One can regard $\mathcal{A}^*$ as the system obtained from $\mathcal{A}$ by adding extra accounts, transactions and balances. This procedure therefore results in a monomorphism from the original system to the augmented system.

(iii) Combining accounts.

The process of combining various sets of accounts in a system $\mathcal{A}$ by forming a quotient system $\mathcal{A}/E$ was described above. It will be shown that there is a canonical homomorphism from $\mathcal{A}$ to $\mathcal{A}/E$ which is surjective. This is a phenomenon which occurs frequently in algebra: given a quotient structure, there is a surjective homomorphism onto it from the original structure. What is important here is the observation that the production of a report on an accounting system is yet another example of a homomorphism.

Let $\mathcal{A} = (A|T|B)$ be an accounting system and let E be an equivalence relation on A. Form the quotient system $\mathcal{A}/E = (A'|T'|B')$. Recall that A' is the set of E-equivalence classes in the natural order induced from A. There is an obvious candidate for a surjective function from A to A', namely $\nu_E : A \to A'$, where $\nu_E(a) = [a]_E$. Thus, ν_E sends an account to its E-equivalence class. In the usual way, there is an induced function $\boldsymbol{\upsilon}_E : \mathrm{Bal}(n) \to \mathrm{Bal}(n')$, the rule being

$$(\boldsymbol{\upsilon}_E(\boldsymbol{\upsilon}))_i = \sum_{a_j E a_i} \upsilon_j .$$

One can see immediately from the definitions of T' and B' that $\boldsymbol{\upsilon}_E(T) = T'$ and $\boldsymbol{\upsilon}_E(B) = B'$. Consequently, one obtains a surjective homomorphism $\boldsymbol{\upsilon}_E : \mathcal{A} \to \mathcal{A}/E$. As mentioned previously, a simple example of a quotient system arises from the identification of all asset, all liability, and all equity accounts after the close of an accounting period. The quotient system then has three accounts. Thus, every accounting system with at least three accounts has a surjective homomorphism onto this three-account system.

(iv) Decomposition of accounts.

An inverse of the operation of combining accounts is provided by an operation in which one or more accounts is split up into several different accounts and its balances and transactions allocated to the new accounts in some prescribed way. If such an operation is applied to a system $\mathcal{A} = (A|T|B)$, there results a new system $\mathcal{A}^* = (A^*|T^*|B^*)$. The relation between the systems is easy to understand, for if the accounts of $\mathcal{A}^*$ are combined again into the original accounts of $\mathcal{A}$ with transaction and balance vectors treated likewise, one recognizes that $\mathcal{A}$ is simply $\mathcal{A}^*/E$. Here, E is the relation in which accounts of $\mathcal{A}^*$ arising from the same account in $\mathcal{A}$ are equivalent. Thus, there is a surjective homomorphism $\mathcal{A}^* \to \mathcal{A}$.

6. The Homomorphism Theorem

There is a well-known theorem in algebra asserting that, when a homomorphism is given from a structure S to another structure S^* of the same type, a certain quotient structure of S is isomorphic with a sub-structure of S^*. Theorems of this type hold for monoids, groups, rings and many other algebraic structures. It will be shown below that accounting systems too display such a theorem, which is called the Homomorphism Theorem for accounting systems. But before this can be stated, it is necessary to introduce a further concept.

Let $\Theta : \mathcal{A} \to \mathcal{A}^*$ be a homomorphism of accounting systems induced by a function $\Theta : \mathcal{A} \to \mathcal{A}^*$. Let $\Theta(A)$ denote the set of distinct accounts $\Theta(a_i)$ in the ordering of $\mathcal{A}^*$, thus $\Theta(A) \subseteq A^*$. The surjective function from A to $\Theta(A)$ sending a_i to $\Theta(a_i)$, denoted by Θ^*, induces $\Theta^* : \mathrm{Bal}(n) \to \mathrm{Bal}(n')$, where $|\Theta(A)| = n'$. Thus, $\Theta^*(T) \subseteq \mathrm{Bal}(n')$, $\Theta^*(B) \subseteq \mathrm{Bal}(n')$. Define the image of Θ to be the accounting system

Im $\Theta = (\Theta(A)|\Theta^*(T)|\Theta^*(B))$. Thus, the accounts, transactions and balances of Im Θ are those inherited from $\mathcal{A}$. Observe that Im Θ is a sub-accounting system of $\mathcal{A}^*$ in the sense of section 4.

Theorem. Let $\Theta : \mathcal{A} \to \mathcal{A}^*$ be a homomorphism of accounting systems arising from a function $\Theta : A \to A^*$. Let E_Θ denote the relation on A in which $a_i E_\Theta a_j$ if and only if $\Theta(a_i) = \Theta(a_j)$. Then E_Θ is an equivalence relation on A. Furthermore, the function ϕ sending the equivalence class of $[a]_{E_\Theta}$ to $\Theta(a)$ gives rise to an isomorphism $\phi : \mathcal{A}/E_\Theta \to$ Im Θ.

Proof. It is clear that E_Θ is an equivalence relation on A, so the quotient system $\mathcal{A}/E_\Theta = (A'|T'|B')$ may be formed as described in section 4. Thus, A' is the set of all E_Θ-equivalent classes. A function $\phi : A' \to \Theta(A)$ is defined by the rule $\phi([a]_{E_\Theta}) = \Theta(a)$. Notice that $\Theta(a)$ depends on $[a]_{E_\Theta}$ and not on the choice of any particular a from this equivalence class. Thus, ϕ is well-defined. It is clear that ϕ is injective, whence ϕ is a monomorphism. Also, ϕ is certainly surjective. To show that ϕ is an epimorphism, one must check that $\phi(T') = \Theta^*(T)$ and $\phi(B') = \Theta^*(B)$ since $\Theta^*(T)$ and $\Theta^*(B)$ are the allowable sets of transactions and balances for Im Θ. Let $\boldsymbol{v}' \in T'$, then $\boldsymbol{v}' = v_{E_\Theta}(\boldsymbol{v})$ for some $\boldsymbol{v} \in T$ by the definition of T'. Now

$$
\begin{aligned}
(\phi(\boldsymbol{v}'))_i &= \phi(v_i') \\
&= \sum_{\Theta(a_j) = \Theta(a_i)} v_j \\
&= (\Theta^*(\boldsymbol{v}))_i .
\end{aligned}
$$

Therefore, $\phi(\boldsymbol{v}') = \Theta^*(\boldsymbol{v})$, which is a typical element of $\Theta^*(T)$. Hence, $\phi(T') = \Theta^*(T)$ and similarly $\phi(B') = \Theta^*(B)$. Therefore, ϕ is an isomorphism. □

Two special cases of the theorem merit attention.

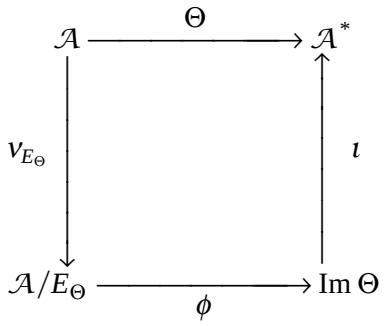
(a) Suppose that Θ is a monomorphism from $\mathcal{A}$ to $\mathcal{A}^*$. In this case, A' is just the set of singletons $\{a_i\}$, so that $\mathcal{A}/E_\Theta$ is essentially the same as $\mathcal{A}$. The Homomorphism Theorem says that $\mathcal{A}$ is isomorphic with Im Θ, which is a sub-accounting system of $\mathcal{A}^*$. Thus, one can think of $\mathcal{A}$ as being embedded as a sub-system of $\mathcal{A}^*$ by Θ.

(b) If Θ is an epimorphism, then Im Θ coincides with $\mathcal{A}^*$ and thus $\mathcal{A}^*$ is isomorphic with $\mathcal{A}/E_\Theta$.

The significance of monomorphisms and epimorphisms is made clear in the following consequence of the Homomorphism Theorem.

Corollary. Every homomorphism of accounting systems may be expressed as the composite of an epimorphism and a monomorphism.

Proof. Let $\Theta : \mathcal{A} \to \mathcal{A}^*$ be a homomorphism. Consider the diagram of accounting systems:



Here, ν_{E_Θ} should be interpreted in the sense of section 4; ϕ is the isomorphism constructed in the proof of the Homomorphism Theorem; and ι is the inclusion monomorphism arising from the inclusion $\Theta(A) \subseteq A^*$. The diagram is commutative, which is to say $\Theta = \iota \cup \phi \cup \nu_{E_\Theta}$. To see this, observe that $\iota\phi\nu_{E_\Theta}(a) = \iota\phi([a]_{E_\Theta}) = \iota(\Theta(a)) = \Theta(a)$ for all $a \in A$. Thus, $\iota\phi\nu_{E_\Theta} = \Theta$. Hence, Θ is the composite of the epimorphism ν_{E_Θ} and the monomorphism $\iota\phi$. □

To appreciate the significance of the corollary, recall that a monomorphism is essentially an embedding of an accounting system in a larger system, while an epimorphism is a passage to a quotient system. Suppose that accounting system $\mathcal{A}^*$ arises from another system $\mathcal{A}$ by any finite sequence of account additions and account combinations. Then the corollary shows that $\mathcal{A}^*$ may be reached from $\mathcal{A}$ by applying *one* combination of accounts and *one* addition of accounts.

7. Accounting systems with finite transaction bases

This section introduces ideas that pertain to the auditability of accounting systems represented by this algebraic framework. The goal here is to be able to limit the set of allowable transactions and balances. This will increase the integrity of the system by filtering erroneous transactions and balances. It also gives auditors a tool with which to check the integrity of systems which were not designed in accordance with this approach. The issue of verification is made even more explicit in section 8. As usual, the approach in both sections is mathematically rigorous.

Let $\mathcal{A} = (A|T|B)$ be an accounting system on a set of n accounts. Then $\mathcal{A}$ has a finite transaction base if one can write $T = T_0 \cup T_1$, where T_0 is a finite subset of Bal(n) and T_1 is the set of all transactions of a certain set of types $\mathbf{T}_1 = \{\mathbf{t}_1, \mathbf{t}_2, \ldots, \mathbf{t}_k\}$.

Thus, an allowable transaction of $\mathcal{A}$ is either one of a specified finite number of transactions or else it is one of a specified type $\mathbf{t}_i$, in which case all transactions of type $\mathbf{t}_i$ are allowable. Such a system may be written

$$\mathcal{A} = (A|T_0|\mathbf{T}_1|B).$$

Proposition. Let $\mathcal{A} = (A|T_0|\mathbf{T}_1|B)$ be an accounting system with finite transaction base. Then the set of allowable transactions is recursive. If in addition $\mathcal{A}$ is bounded, then it is a recursive system.

Proof. Let $\mathbf{T}_1$ consist of all transactions of types $\mathbf{t}_1, \mathbf{t}_2, \ldots, \mathbf{t}_k$ and let $T = T_0 \cup \mathbf{T}_1$ be the set of allowable transactions. If $\boldsymbol{\upsilon}$ is any balance vector, there is a recursive procedure to determine if $\boldsymbol{\upsilon} \in T$. In the first place, one can check to see if $\boldsymbol{\upsilon} \in T_0$ since T_0 is finite. Secondly, one can compute the type of $\boldsymbol{\upsilon}$ and determine if it equals one of the $\mathbf{t}_i$. Thus, one can decide if $\boldsymbol{\upsilon}$ belongs to T, so T is recursive. If, additionally, $\mathcal{A}$ is bounded, then B is finite and hence recursive. Thus, $\mathcal{A}$ is a recursive system. □

It seems plausible that any realistic accounting system will have a finite transaction base. An example of a specific allowable transaction might be a mortgage or other fixed payment. On the other hand, a debit to cash and a credit to accounts receivable is an example of an allowable transaction type in which any amounts are permitted. This allowable type would have a "+" in cash and a "–" in accounts receivable and "0"s elsewhere. For a bounded accounting system, the allowable transactions can be represented as either specific transactions and allowable types with associated bounds or, equivalently, as a listing of specific transactions only. This is true since the finite sets of bounded allowable types can be listed as finite sequences of specific transactions and the finite union of finite sets remains a finite set. This representation would therefore remain a recursive system.

On the other hand, there are accounting systems which do not have a finite transaction base. An example is the hypothetical three-account system $\mathcal{A}$ in which B = Bal(3) and T is the set of all transaction vectors of the form

$$\begin{bmatrix} 1 \\ 1+x \\ -2-x \end{bmatrix}, \quad x = 0, 1, 2, \ldots \ .$$

Notice that all vectors in T are of the type

$$\begin{bmatrix} + \\ + \\ - \end{bmatrix}.$$

Thus, if $\mathcal{A}$ had a finite transaction base, T would have to contain all vectors of the above type. However, T does not contain the transaction

$$\begin{bmatrix} 2 \\ 2 \\ -4 \end{bmatrix},$$

hence $\mathcal{A}$ does not have a finite transaction base.

If an accounting system has only finitely many allowable transactions, the monoid of the system will be finitely generated. However, this need not be true if the system merely has a finite transaction base. Indeed, let $\mathcal{A}$ be the three-account system with $T_0 = \{\ \}$, T_1 = all transactions of the type in the above example and B = Bal(3). If the monoid of $\mathcal{A}$ were finitely generated, say by the vectors

$$\begin{bmatrix} a_i \\ b_i \\ -a_i - b_i \end{bmatrix},$$

where $a_i > 0$, $b_i > 0$, it could not contain, for example, the vector

$$\begin{bmatrix} 1 \\ h \\ -1-h \end{bmatrix},$$

where h is greater than all of the b_i.

7.1. Finitely specified accounting systems

In addition to having a finite transaction base, a realistic accounting system will most likely have finite upper and lower bounds, m_i and M_i, for the balance of its ith account for all accounts of the system. These bounds can be thought of as $n \times 1$ column vectors m and M, where $m_i \leq M_i$. A realistic system will also have certain allowable quotient systems corresponding to reports that the system could reasonably be required to generate. Thus, an n-account system will have a list of allowable equivalence relations on the set $\{1, 2, \ldots, n\}$. These considerations suggest the following type of accounting system as being the closest to a realistic system of the systems considered here.

An accounting system $\mathcal{A}$ on n accounts is said to be finitely specified if it has a finite transaction base, is bounded and is associated with a set of equivalence relations on the set of accounts. An appropriate notation for $\mathcal{A}$ is the sextuple

$$\mathcal{A} = (A|T_0|\mathbf{T}_1|m|M|E),$$

where A is the set of accounts, T_0 is a finite set of specific allowable transactions, $\mathbf{T}_1$ is a set of allowable transaction types, m and M are the lower and upper bound vectors, and E is a set of equivalence relations on $\{1, 2,\ldots, n\}$. An attractive feature of such a finitely specified system is the obvious possibility of machine implementation.

In order to specify $\mathcal{A}$ for the purpose of machine implementation, one must specify:

(i) a finite subset T_0 of Bal(n);

(ii) a finite set $\mathbf{T}_1$ of $n \times 1$ column vectors with entries -1, 0, $+1$ corresponding to type vectors with entries $-$, 0, $+$;

(iii) two $n \times 1$ column vectors m and M over $\mathbb{Z}$ such that $m_i \leq M_i$;

(iv) a finite list E of equivalence relations on $\{1, 2,\ldots, n\}$ described by either partitions or n-square boolean matrices.

In principle, it is a straightforward matter to write a piece of computer code which will incorporate the above data and the rules for forming new balances, and so can operate the accounting system $\mathcal{A}$.

8. Verification of balances

Consider an accounting system $\mathcal{A} = (A|T|B)$ on n accounts. Suppose that $\boldsymbol{b}(0)$ is the balance vector of $\mathcal{A}$ at some point of time. One can ask of an arbitrary vector $\boldsymbol{b}$ in Bal(n) whether it could be the balance vector of $\mathcal{A}$ at some future time. One question is thus whether $\boldsymbol{b}$ could be obtained by applying a finite sequence of transactions in T to $\boldsymbol{b}(0)$. One can further inquire if there is such a sequence which produces intermediate balance vectors that all belong to B. Accounting systems for which there are procedures to answer these questions will have a capability to verify final balances at the end of an accounting period. Systems with a finite transaction base are of this type, as the following result shows.

Theorem. Let $\mathcal{A} = (A|T_0|\mathbf{T}_1|B)$ be an accounting system with a finite transaction base. Let $\boldsymbol{b}(0)$and $\boldsymbol{b}$ be two balance vectors on A.

(a) There is a recursive procedure to decide if there is a finite sequence of allowable transactions of $\mathcal{A}$ which, when applied to $\mathcal{A}$ with initial balance $\boldsymbol{b}(0)$, will yield $\boldsymbol{b}$.

(b) If B is finite, there is a recursive procedure to decide if there is a sequence as in (a) which does not produce any non-allowable intermediate balances at any stage.

Proof. (a) Let $T_0 = \{u_1, u_2,\ldots, u_r\}$ and let $\mathbf{T}_1$ be the set of all vectors of types $\mathbf{t}_1, \mathbf{t}_2,\ldots, \mathbf{t}_k$. Let $y^{(i)}$ be a general vector of type $\mathbf{t}_i$ with non-zero entries $\pm y^{(i)}_{j,k}$, where $y^{(i)}_{j,k} > 0$. There

will be a sequence of allowable transactions producing $\boldsymbol{b}$ from $\boldsymbol{b}(0)$ if and only if there are integers $x_i > 0$ and $y_{j,k}^{(i)} > 0$ such that

$$\sum_{i=1}^{r} x_i \boldsymbol{u}(i) + \sum_{i=1}^{k} y^{(i)} = \boldsymbol{b} - \boldsymbol{b}(0).$$

This is equivalent to a system of n linear equations over $\mathbb{Z}$ in the unknowns $\boldsymbol{u}(i)_j$ and $y_{j,k}^{(i)}$. Whether such a solution exists is a problem of integer programming. Algorithms exist to decide if such solutions exist, the best known of these is Gomery's Algorithm.

(b) Now assume that B is finite. The problem is more difficult in this case since all intermediate balance vectors produced by the sequence of allowable transactions must remain in B. Thus, the order of application of the transactions in the sequence is significant. The crucial observation is that if there is such a sequence of allowable transactions leading from $\boldsymbol{b}(0)$ to $\boldsymbol{b}$, which preserves allowable balances, then there is one of length less than or equal to $d = |B|$.

To see this, assume that there is a shortest sequence of this type with length $h > d$. Consider the successive balances computed as one moves along the sequence. All of these balances are in B, hence the $(d + 1)$th balance must have already occurred, say at stage i, where $i \leq d$. Since the ith and $(d + 1)$th balance vectors are identical, the allowable transactions at steps $i + 1$, $i + 2, \ldots, d$ can be deleted without changing the essential properties of the sequence. But now one has a sequence of length less than h leading to b, which is a contradiction to the choice of the sequence.

It follows that if there is a sequence from $\boldsymbol{b}(0)$ to $\boldsymbol{b}$, there is one of length at most d. The number of possible forms of such a sequence is at most $(|T_0| + k)^d$, for at each point of the sequence one has either an element of T_0 or a general transaction of one of the k types $\mathbf{t}_i$. Each such sequence gives rise to a linear system in unknowns $y_{j,k}^{(i)}$ as in (a). To this system must be added the conditions that the intermediate balances remain in B. The possible intermediate balances for such a sequence number at most $(d - 1)^d$. Each choice of intermediate balances involves the addition of a further $(d - 1)$ $(n - 1)$ equations at most. In summary, to decide the question one must check each of a finite number of linear systems over $\mathbb{Z}$ for positive integral solutions. Again, this is a problem of integer programming. □

Thus, a precise verification of balances is possible for finitely specified accounting systems, although it is clear that the amount of computations required by the second algorithm is very large. If a check of intermediate balances is not required, then balance verification is possible in any accounting system with a finite transaction base, and here the algorithm is much more efficient.

9. Conclusions

This work has described a useful algebraic structure for the representation of accounting systems. The use of vectors to contain the accounting data for transactions

and balances in the system is the starting point of this new structure. This allows the system to be abstractly conceived of as a group or semi-group and also induces the monoid space within which the system operates. The introduction of the concept of partitioning the accounts and providing homomorphisms to control the generation of reports gives the system an ability to represent most of the manipulations which occur in realistic accounting systems. In addition, providing a description of the accounting system in terms of abstract algebra provides convenient parallels into the theories of Turing machines and computability.

Future research by the authors is proceeding in three general directions. The first is to add elements to the description which will adequately provide for concepts of managerial control to be given through the algebraic structure. This will bring the abstract structure closer to real-world accounting systems. The second area of continuing research is a graph theoretic description of the accounting system as a series of flows to and from the different accounts. This research promises to provide some interesting insights into the problems of storage of accounting data in computerized accounting systems. The final research direction involves methods to be used in the comparison of accounting systems which are represented in the manner described in this paper. These comparisons will provide a richer structure for the comparison of information systems generally, as Nehmer [3] shows.

References

[1] M.A. Arbib, *Theories of Abstract Automata*, Prentice-Hall, Englewood Cliffs, NJ, 1969.
[2] T.C. Hu, *Integer Programming and Network Flows*, Addison-Wesley, Reading, MA, 1970.
[3] R.A. Nehmer, *Accounting Information Systems as Algebras and First Order Axiomatic Models*, University Microfilms International, Ann Arbor, MI, 1988.
[4] R. Mattessich, Towards a general and axiomatic foundation of accountancy, Accounting Research (October 1957) 328–355.
[5] R. Mattessich, *Accounting and Analytical Methods*, Irwin, Homewood, IL, 1964.
[6] K.A. Ross and C.R.B. Wright, *Discrete Mathematics*, Prentice-Hall, Englewood Cliffs, NJ, 1985.